A FORMULA CHEAT SHEET / OPTICOM

Prefix Space Partitioning.

Six formulas that describe how a combinatorial search space fractures into thousands of independent micro-tasks — and why those tasks are wildly, mathematically unequal.

NOTATION

n	size of the universe N	r	size of each combination	$c_1 < \cdots < c_r$	elements, strictly increasing
D	prefix depth, $1 \le D < r$	c_D	anchor — last fixed element	$W(c_D)$	workload of a single task

01 The Ceiling Limit

LEXICOGRAPHICAL BOUNDS

In strict lexicographical order, each element c_i of a valid r-combination is capped by a position-dependent ceiling. The further left in the sequence, the lower the ceiling.

$$1 \le c_1 < c_2 < \cdots < c_r \le n \quad \text{and} \quad c_i \le n - r + i$$

02 What Is a Task?

PREFIX + ITERATED TAIL

Fix a depth D with $1 \le D < r$. A task is the entire set of combinations that share a single constant prefix of length D — the worker thread iterates only over the remaining r − D positions.

$$(c_1, c_2, \ldots, c_D)$$

THE FIXED PREFIX

$$(c_{D+1}, \ldots, c_r)$$

THE ITERATED TAIL

03 How Many Tasks?

CARDINALITY OF THE PARTITION

Valid depth-D prefixes are exactly the D-subsets of the restricted pool {1, 2, ..., n − r + D}. The total task count is therefore a single binomial:

$$T_{\text{tasks}} = \binom{n-r+D}{D}$$

EXAMPLE · C(30,15) with D = 5 C(20,5) = 15,504 independent tasks.

04 Workload of One Task

DEPENDS ONLY ON THE ANCHOR c_D

Here is the surprise: a task's workload is completely independent of its first D − 1 prefix elements. Only the anchor c_D matters, because the tail is drawn from the pool $\{c_D+1, \ldots, n\}$, of size $n - c_D$.

$$W(c_D) = \binom{n-c_D}{r-D}$$

05 The Extreme Asymmetry

HEAVIEST BRANCH VS LIGHTEST BRANCH

Lexicographical trees lean drastically left. Prefixes starting with small numbers contain exponentially more combinations than prefixes starting with large ones. Substituting the endpoints of c_D's range produces the two extremes:

HEAVIEST BRANCH

When the anchor sits at its minimum, $c_D = D$, the prefix is (1, 2, ..., D) and the task explodes:

$$W_{\max} = \binom{n-D}{r-D}$$

Maximum possible workload.

LIGHTEST BRANCH

When the anchor sits at its ceiling, $c_D = n - r + D$, only one tail is possible — the task terminates after a single combination:

$$W_{\min} = \binom{r-D}{r-D} = 1$$

A single combination.

For C(30, 15) at depth D = 5: the heaviest task — prefix (1, 2, 3, 4, 5) — generates 3,268,760 combinations. The lightest generates one. A variance of millions to one within the same partition.

06 Worked Example

C(7, 5) · DEPTH D = 3

With n = 7, r = 5, D = 3 the anchor c_3 ranges over {3, 4, 5}. The workload formula reduces to:

$$W(c_3) = \binom{7 - c_3}{2}$$

ANCHOR c_3	WORKLOAD $W(c_3)$	# OF PREFIXES	COMBOS	% OF SPACE
3	$\binom{4}{2} = 6$	1	6	28.6%
4	$\binom{3}{2} = 3$	3	9	42.9%
5	$\binom{2}{2} = 1$	6	6	28.6%
TOTAL	—	**10 tasks**	**21**	**100%**

One task (the prefix 1,2,3) absorbs nearly 30% of the entire computational workload. This is precisely why static block scheduling fails — and why work-stealing schedulers (OpenMP's schedule(dynamic)) are an architectural necessity, not an optimization.

07 At a Glance

ALL SIX FORMULAS IN ONE PLACE

QUANTITY	FORMULA	CONDITION
Ceiling for c_i	$c_i \le n - r + i$	*per-position bound*
Total tasks	$\binom{n - r + D}{D}$	*granularity of partition*
Workload per task	$\binom{n - c_D}{r - D}$	*depends only on* c_D
Heaviest task	$\binom{n - D}{r - D}$	$c_D = D$
Lightest task	1	$c_D = n - r + D$
Total combinations	$\binom{n}{r}$	*sanity check · sum of all* $W(c_D)$